GUÍA DE LABURO: TDAs, PILAS Y COLAS

1. Intro: ¿Qué onda con la Abstracción?

¡Buenas, gente! Bienvenidos a la segunda clase. Hoy vamos a bajar a tierra ese concepto que suena re místico pero que es el pan de cada día si quieren dejar de ser "picacodigo" y ser desarrolladores de verdad: la **Abstracción** y los **TDAs (Tipos de Datos Abstractos)**.

Para entender la abstracción, piensen en un **auto**. Si querés ir de acá a la facultad, te sentás, ponés primera y salís andando. Sos un **Usuario** del auto: te interesa el "qué hace" (te lleva) y no el "cómo lo hace". Ahora, si sos el mecánico, tenés que meter mano en el motor, entender los pistones y la inyección. Ahí bajás de nivel, sos el **Implementador**.

Otra analogía clásica que está en las fuentes es la del **Gato**:

- La **abuela** ve una bola de pelos que hace "purr purr" y le da comida (Abstracción de alto nivel, puro amor).
- El **veterinario** ve un sistema complejo con riñones, pulmones y un corazón (Bajo nivel de abstracción, conoce los órganos y la anatomía).

Ojo acá, pónganse rigurosos: En programación, **el TDA** se define a través de una **interfaz**, que es una **clase abstracta** que contiene la especificación de los métodos (qué puede hacer el bicho). Separamos la especificación (el contrato) de la implementación (el código sucio). Esto nos permite que las implementaciones sean intercambiables: podés cambiar todo el motor por uno eléctrico y, si el volante y los pedales funcionan igual, el usuario ni se entera.

---

2. Actividad 1: "El Contrato del Millón" (60 min)

Acá es donde se ponen la gorra de arquitectos. Vamos a simular un negocio real para entender la diferencia entre Interfaz e Implementación.

El Problema

Un cliente millonario necesita una estructura para manejar datos. El tipo no sabe un pomo de código, pero te firma un cheque con muchos ceros si cumplís con el comportamiento que él pide. Él solo ve la **Caja Negra**.

Fase de Contrato (Interfaz)

Antes de tirar una línea de código, hay que firmar la **Especificación**. Estos métodos son sagrados. Según nuestra fuente técnica, los contratos obligatorios son:

- **Si eligen Pila (Stack):** `InicializarPila`, `Apilar`, `Desapilar`, `Tope` y `PilaVacia`.
- **Si eligen Cola (Queue):** `InicializarCola`, `Acolar`, `Desacolar`, `Primero` y `ColaVacia`.

Simulación de Caja Negra

Divídanse en dos bandos:

1. **Los Usuarios (Alto Nivel):** Ustedes solo usan los métodos del contrato. No saben si la data se guarda en un arreglo o si hay un duende anotando en un papel.
2. **Los Implementadores (Bajo Nivel):** Ustedes escriben la lógica oculta. Tienen que asegurar que el comportamiento sea el esperado para cualquier estado posible.

3. Actividad 2: "Guerra de Estrategias"

Ahora nos ensuciamos las manos con implementaciones estáticas. No todas las formas de guardar datos cuestan lo mismo. Piensen en la eficiencia como la diferencia entre tener la ropa tirada en una **silla (L1 Cache)** vs. buscarla en el **fondo del placard (Plato giratorio)**.

El Desafío

Analicen estas 3 estrategias de implementación de Pilas usando arreglos:

1. **Estrategia 1:** Arreglo + variable `cantidad` (que apunta a la **primera posición libre**).
2. **Estrategia 2:** Arreglo + variable `cantidad`, pero el **Tope siempre es la posición 0**. Cada vez que apilás, tenés que "correr" todos los elementos hacia la derecha.
3. **Estrategia 3:** Arreglo donde la **posición 0 guarda la cantidad de elementos**. El tope es la última posición ocupada.

Tabla Comparativa de Costos

Estrategia	¿Hay que desplazar elementos?	Pro (Lo bueno)	Contra (El bardo)
1 (Var. externa)	No	Es rapidísima. O(1).	Necesitás una variable de control afuera del arreglo.
2 (Tope en 0)	**SÍ**	El tope está siempre a mano al inicio.	Es **lentísima**. Desplazar elementos es caro en tiempo de CPU.
3 (Pos. 0 = Cant)	No	Ahorrás variables de control	Perdés un lugar: la capacidad usable es **N-1**. Ojo con los

		externas.	`IndexOutOfBounds`.

---

4. Resumen de Conceptos Clave (LIFO vs. FIFO)

Para que no pasen papelones en los parciales, acá tienen la posta:

- **Pila (LIFO - Last In, First Out):** El último que entra es el primero que sale. Como la pila de platos: el último que apoyaste es el primero que agarrás para lavar.
- **Cola (FIFO - First In, First Out):** El primero que llega es el primero que sale. Como la fila en la ventanilla del banco: el que llegó primero, se atiende primero.
- **Cola con Prioridad:** Acá la cosa cambia. El orden depende de la **prioridad asociada**. Los métodos obligatorios son:
    - `AcolarPrioridad(valor, prioridad)`: Metés el dato con su "chapa".
    - `Prioridad()`: Te dice qué tan importante es el que está primero.
    - *Nota:* Si tienen la misma prioridad, el implementador decide el orden (semántica libre).

Actividad 1: "El contrato del Millón"

Bando 1 – Los Usuarios (Alto Nivel)

Los usuarios trabajan únicamente con la interfaz del TDA y utilizan la estructura como una "caja negra". Estos solo conocen las operaciones disponibles y no saben cómo se almacenan los datos internamente ni tampoco acceden a arreglos, nodos ni variables internas. A los usuarios les interesa únicamente el comportamiento del TDA

En una Pila, por ejemplo, los usuarios solo utilizan:

- *InicializarPila*
- *Apliar*
- *Desapilar*
- *Tope*
- *PilaVacia*

La ventaja principal es que pueden usar la estructura sin preocuparse por la implementación interna.

Bando 2 – Los Implementadores (Bajo Nivel)

Los implementadores desarrollan la lógica interna que permite que el TDA funcione correctamente. Diseñan la estructura interna y deciden como almacenar los datos. También controlan memoria, índices, nodos y validaciones. Son los que garantizan que las operaciones del contrato funcionen correctamente en cualquier situación.

Los implementadores pueden crear distintas implementaciones mediante arreglos, listas enlazadas o estructuras dinámicas o estáticas.

La principal responsabilidad del implementador es mantener el comportamiento definido por la interfaz, independientemente de cómo se resuelva internamente.

Actividad 2: "Guerra de estrategias"

Estrategia 1 – Arreglo + variable cantidad externa

Esta implantación utiliza un arreglo para almacenar datos y una variable externa que indica la cantidad de elementos o la próxima posición libre

Características:

- El tope se ubica al final de los elementos cargados
- No es necesario desplazar elementos
- Las operaciones Apliar y Desapilar tienen costo O(1)

Si bien es muy rápida y posee una implementación eficiente, requiere una variable de control externa al arreglo.

Estrategia 2 – Tope en posición 0

La implementación mantiene el tope siempre en la posición 0 del arreglo. Cada inserción obliga a desplazar todos los elementos una posición hacia la derecha. Esta operación Aplicar tiene costo O(n).

El tope siempre esta inmediatamente accesible pero el desplazamiento constante vuelve la implementación lenta con un alto costo temporal.

Estrategia 3 – Posición 0 almacena cantidad

La implementación utiliza la posición 0 del arreglo para guardar la cantidad de elementos. Los datos comienzan desde la posición 1 y el tope se encuentra en la última posición ocupada. No requiere una variable externa ni desplazar elementos.

Su desventaja mas grande es que se pierde una posición útil del arreglo y requiere cuidado con los limites del arreglo para evitar errores de índice.